

30/04/2025

Rapport : Projet Programmation Mobile I 2024- 2025



Auteur : Osman Ziylan
ENSEIGNANT : JONATHAN RIGGIO

Table des matières

Introduction.....	4
Description des technologies utilisés.....	5
Firebase Authentication	5
Spotify API.....	5
Gestion des visuels avec Picasso	5
Google Maps API.....	5
Système de notifications	6
Mise en œuvre technique :	6
Avantages :	7
Cloud Firestore.....	7
Fonctionnalités	8
Intégration de Cloud Firestore.....	8
Connexion et gestion des utilisateurs	8
Connexion persistante des utilisateurs	8
Ajout et gestion des musiques	9
Gestion centralisée avec FirestoreRepository.....	10
Avantages principaux :	10
Jouer une musique via l'application.....	10
Redirection vers YouTube	11
Like des musiques	11
Cartographier les genres musicaux	12
Réinitialisation automatique de la playlist	12
Mise à jour en temps réel de la playlist	12
Classement dynamique des musiques par nombre de likes	12
Analyse.....	13
Architecture globale	13
Interactions utilisateur : Cas concret du like.....	13
Gestion des tâches périodiques	14
Relations entre les couches de l'architecture	14
Vue d'ensemble avec le diagramme de classes principal.....	14
Vue d'ensemble avec le diagramme de classes principal.....	15
UX/UI	16
ImageButtons pour une interface intuitive.....	16

SearchView et en-tête personnalisée	16
Bouton retour dans MusicActivity et MapsActivity.....	16
Légende dans MapsActivity.....	16
Mode jour/nuit automatique	17
Système de like interactif	17
Feedback de l'étudiante.....	17
Améliorations après le feedback de l'étudiante	17
Révision des couleurs dans la partie connexion.....	17
Améliorations apportées à MainActivity et MusicActivity	18
Limitations et développement futur	19
Limitations.....	19
Connexion Internet.....	19
Limitations des API	19
Recommandations et corrections dans la recherche musicale :	19
Lecture de musique intégrée :.....	19
Développements futurs	20
Proposition de musiques aléatoires sur MusicActivity :.....	20
Notification de la musique la plus votée avant réinitialisation	20
Affichage de la musique de la semaine ou du mois	20
Exploration d'une solution avec une heatmap.....	20
Conclusion	21
Liens YouTube vers les vidéos	22
Vidéo de démonstration.....	22
Vidéo Bonus	22

Rapport du projet Musio

Introduction

Le projet **Musio**, développé pour l'année académique 2024-2025, s'inspire de concepts populaires comme **Snapchat** et **Instagram** pour créer une expérience collaborative unique. Il offre aux utilisateurs la possibilité de contribuer à une playlist en ligne, où chacun peut ajouter ses morceaux favoris. L'application Android Musio cible spécifiquement les étudiants et les enseignants du campus "**La Plaine**", favorisant ainsi le partage musical et l'interaction autour des goûts communs.

Parmi les **fonctionnalités essentielles**, on trouve :

- **Adaptation au changement d'orientation de l'appareil** : Transitions fluides entre les modes portrait et paysage.
- **Gestion de la persistance des données** : Les informations clés, telles que les comptes utilisateurs ou les playlists, sont conservées même après la fermeture de l'application.
- **Système d'authentification via Firebase** : Chaque utilisateur doit pouvoir s'inscrire et se connecter de manière sécurisée pour participer activement à la communauté.
- **Notifications utilisateurs** : L'application envoie des notifications toutes les 24h pour informer que la playlist du jour est disponible
- **Accès à des données en ligne** : Utilisation de l'API musicale Spotify pour récupérer les titres, artistes et genres des morceaux proposés.
- **Utilisation de cartes interactives** : Intégration de Google Maps pour visualiser la répartition des genres musicaux sur le campus.

Ce rapport vise à présenter une **vue complète** du projet en abordant les points suivants :

- **Description des technologies utilisées** : Présentation des technologies employées et justification de leur choix.
- **Fonctionnalités** : Description des différentes fonctionnalités de l'application, incluant leur implémentation et leur utilité pour les utilisateurs, ainsi que les choix technologiques et les défis rencontrés.
- **Analyse** : Évaluation des performances de l'application, de son efficacité, de sa stabilité et de sa convivialité, intégrant les retours des utilisateurs et des aspects UX/UI.
- **Limitations** : Mise en lumière des contraintes et des limites rencontrées, ainsi que des aspects à améliorer pour offrir une meilleure expérience utilisateur.
- **Développements futurs** : Recommandations pour les futures améliorations et évolutions de l'application, basées sur les leçons tirées de ce projet et les suggestions des utilisateurs.

Musio s'impose comme une plateforme innovante qui encourage le partage musical et enrichit les échanges culturels sur le campus. Ce rapport détaille les aspects techniques, fonctionnels et créatifs qui ont façonné son développement.

Description des technologies utilisés

Firebase Authentication

Firebase Authentication, une solution proposée par Google, a été imposée par l'énoncé du projet pour garantir un système d'inscription et de connexion sécurisé. Cette technologie permet aux utilisateurs de créer et gérer leurs comptes via des identifiants tels qu'une adresse e-mail et un mot de passe. En cas d'oubli de mot de passe, Firebase offre une fonctionnalité pratique : l'envoi d'un lien de réinitialisation par e-mail, simplifiant ainsi la récupération d'accès pour les utilisateurs.

Dans le cadre de **Musio**, Firebase Authentication joue un rôle essentiel en assurant un accès sécurisé à la plateforme. Elle constitue la première étape pour permettre aux utilisateurs de participer activement tout en garantissant la protection de leurs données personnelles.

Spotify API

L'API Spotify est intégrée dans Musio pour récupérer les informations sur les morceaux ajoutés par les utilisateurs. Ces données incluent l'ID du morceau, son titre, l'artiste, ainsi que son genre musical. Ces métadonnées enrichissent les playlists et permettent de personnaliser l'expérience utilisateur en organisant efficacement les morceaux.

Gestion des visuels avec Picasso

Bien que Spotify fournisse des métadonnées complètes, les images comme les pochettes d'album sont prises en charge par **Picasso**, une bibliothèque Android spécialisée dans la gestion d'images.

Cette séparation garantit une gestion efficace des visuels tout en réduisant la dépendance à l'API Spotify. La combinaison de Spotify et Picasso permet à Musio de présenter des morceaux de manière professionnelle et attrayante avec des données riches et des visuels dynamiques.

Google Maps API

La carte dans **Musio** sert à géolocaliser les musiques grâce à des marqueurs colorés représentant les genres musicaux, tout en offrant aux utilisateurs la possibilité de se géolocaliser eux-mêmes. Cette fonctionnalité enrichit l'interactivité de l'application et renforce son aspect communautaire.

Initialement, Mapbox avait été envisagé pour son légèreté, sa fluidité et son esthétique moderne. Cependant, lors de la phase de développement en Android Java, il s'est avéré impossible d'implémenter l'affichage des marqueurs. Bien que cela soit réalisable en Kotlin, l'énoncé spécifie clairement que le projet doit être développé en Java. Par conséquent, changer de langage aurait non seulement été non conforme aux consignes, mais aurait également complexifié la structure globale du projet.

Un second test a été effectué avec OpenStreetMap, qui répondait aux besoins fonctionnels. Toutefois, son graphisme peu travaillé et la difficulté de visualiser les marqueurs de manière claire et intuitive ont rendu cette solution moins adaptée.

C'est pour ces raisons que le choix final s'est porté sur **Google Maps API**, qui offre :

- ✓ **Graphismes professionnels** : Une interface esthétique et conviviale facilitant la navigation.
- ✓ **Flexibilité** : Des outils puissants pour personnaliser les marqueurs et gérer les données géographiques.
- ✓ **Fiabilité** : Une intégration robuste avec Android assurant une expérience utilisateur fluide.

Ainsi, Google Maps s'est imposé comme la solution optimale pour satisfaire les exigences visuelles et techniques de **Musio**.

Système de notifications

Les notifications jouent un rôle clé en maintenant les utilisateurs informés des nouvelles playlists disponibles, favorisant l'engagement et la participation active. Musio utilise le système de notifications natives d'Android pour informer les utilisateurs que la nouvelle playlist collaborative est disponible. Cette fonctionnalité repose sur la classe **NotificationUtils**, développée en s'inspirant de la documentation officielle Android Developers¹.

Mise en œuvre technique :

Création d'un **canal de notifications** : Depuis Android 8 (Oreo), les canaux de notifications sont obligatoires pour gérer les paramètres de notification. Un canal nommé "Musio Notifications" est configuré pour organiser les notifications liées à l'application² :

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    NotificationChannel channel = new NotificationChannel(
        CHANNEL_ID,
        CHANNEL_NAME,
        NotificationManager.IMPORTANCE_DEFAULT
    );
    channel.setDescription(CHANNEL_DESC);
}
```

Affichage des notifications³ : Lors de la réinitialisation quotidienne de la playlist, une notification est générée avec un titre, un message et une icône personnalisée. Un **PendingIntent**⁴ redirige l'utilisateur vers l'écran principal de l'application :

```
NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_stat_name)
    .setContentTitle(title)
    .setContentText(message)
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .setContentIntent(pendingIntent)
    .setAutoCancel(true);
```

¹ Le lien vers la documentation pour les notifications : [Create and manage notification channels | Views | Android Developers](#)

² Ce code provient de : « ...\\musio\\main\\utils\\NotificationsUtils » L : 27

³ Toujours dans NotificationsUtils L : 69

⁴ PendingIntent permet à une application Android de déléguer une action à une autre application ou au système, pour qu'elle s'exécute plus tard, même si l'application d'origine est fermée

Avantages :

- ✓ **Compatibilité étendue** : Fonctionne sur les versions d'Android à partir de Lollipop (API 21) jusqu'à la dernière version, Android 13.
- ✓ **Expérience utilisateur optimisée** : La notification, discrète mais informative, permet à l'utilisateur d'accéder directement à l'application en un clic.
- ✓ **Sécurité et performance** : L'utilisation de `PendingIntent.FLAG_IMMUTABLE` renforce la sécurité sur Android 12+, et la gestion des notifications suit les bonnes pratiques recommandées par Google.

Ce système garantit une communication efficace avec les utilisateurs, tout en exploitant des fonctionnalités modernes et robustes d'Android.

Cloud Firestore

Pour répondre aux besoins de **Musio**, Cloud Firestore a été choisi pour stocker et organiser les données clés de l'application. Cette base de données propose une structure flexible et adaptée à la gestion des fonctionnalités interactives de l'application, tout en assurant une synchronisation en temps réel.

Les données sont structurées autour de deux principaux documents :

- **Document des utilisateurs (« users »)** : Contenant des informations essentielles comme les identifiants uniques et les noms des utilisateurs, nécessaires pour leur gestion et leur interaction dans l'application.
- **Document des musiques (« proposals »)** : Regroupant des informations sur les morceaux partagés, telles que leur ID, leur titre, leur artiste, les coordonnées de l'utilisateur et des éléments visuels comme l'URL de l'image associée.

Ce choix permet de centraliser les données des musiques et de leurs interactions avec la carte (géolocalisation des morceaux), ainsi que les informations utilisateur, créant ainsi un environnement cohérent et fluide.

Pourquoi Cloud Firestore ?

Cloud Firestore s'est imposé grâce à :

- ✓ **Synchronisation en temps réel**, essentielle pour permettre une mise à jour immédiate des playlists collaboratives.
- ✓ **Structure adaptative**, facilitant la gestion des données complexes comme la géolocalisation.
- ✓ **Évolutivité**, assurant un fonctionnement fluide même avec une augmentation du nombre d'utilisateurs et du volume de données.

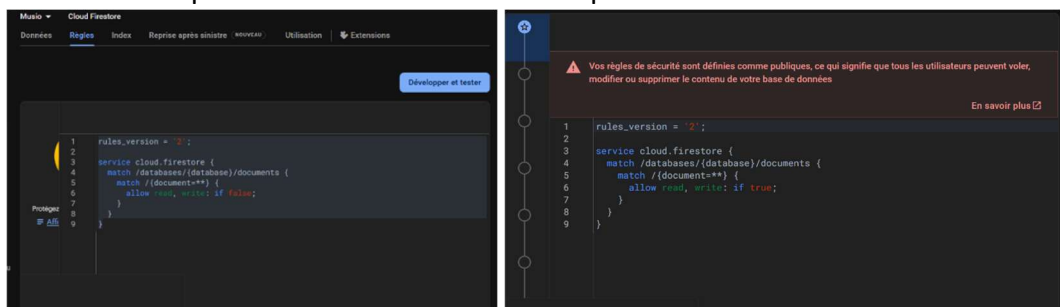
De plus, Cloud Firestore facilite le lien entre les musiques et les utilisateurs, en offrant une gestion efficace des relations entre ces entités. Cela garantit une expérience utilisateur fluide et organisée tout en simplifiant le stockage et la récupération des données.

Fonctionnalités

Intégration de Cloud Firestore

Description : Cloud Firestore est utilisé comme base de données principale pour gérer le stockage et la récupération des données de l'application. Il permet de conserver des informations telles que les playlists, les utilisateurs et leurs interactions dans un format structuré et sécurisé. Son choix s'appuie sur sa flexibilité, ses capacités de synchronisation en temps réel, et sa simplicité d'intégration, qui répondent parfaitement aux besoins d'une application dynamique.

Défis rencontrés et solutions apportées : Lors de l'intégration, un problème majeur concernait les autorisations par défaut dans la console Firebase, définies sur « false », ce qui empêchait l'envoi des données. Une investigation approfondie m'a permis de découvrir, avec l'aide de ressources comme ChatGPT, qu'il fallait modifier ces autorisations en « true ». Bien que ce problème ait été résolu, il a révélé certaines limites de la documentation Firebase, qui ne fournissait pas d'indications claires sur ce point :



Bien que ce problème ait été résolu, il a mis en lumière certaines limites de la documentation, qui n'a pas fourni d'indications claires sur ce point.

Connection et gestion des utilisateurs

Description : Les utilisateurs peuvent s'inscrire et se connecter via Firebase Authentication, garantissant un accès sécurisé et personnalisé à leurs comptes. Une option de réinitialisation de mot de passe est également incluse pour simplifier la récupération en cas d'oubli. Cette fonctionnalité assure une expérience utilisateur fluide tout en protégeant les données personnelles des utilisateurs grâce aux robustes mécanismes de sécurité de Firebase.

Défis rencontrés et solutions apportées : Un défi majeur a été rencontré lorsque l'API refusait l'utilisation des services de Firebase Firestore, bloquant ainsi l'intégration. Pour résoudre ce problème, un nouveau compte a été créé, permettant d'utiliser pleinement les fonctionnalités de Firebase Authentication. Ce processus a également renforcé la gestion des dépendances et la configuration de l'application.

Connexion persistante des utilisateurs

Description : Cette fonctionnalité permet de maintenir les utilisateurs connectés après leur authentification, offrant une expérience fluide et évitant la nécessité de se reconnecter à chaque ouverture de l'application. Dans l'activité principale (MainActivity), la connexion de

⁵ Section règles de Cloud Firestore [musio – Cloud Firestore – Règles – Console Firebase](#)

l'utilisateur est vérifiée dès le lancement grâce au code suivant⁶ :

```
// Authentication check
FirebaseUser currentUser = FirebaseAuth.getInstance().getCurrentUser();
if (currentUser == null) {
    redirectToHome();
    return;
}
```

Si l'utilisateur n'est pas connecté, il est redirigé vers « HomeActivity », qui gère le processus d'authentification.

Défis rencontrés et solutions apportées : Le défi majeur était de gérer efficacement la pile d'activités lors de la redirection pour éviter les chevauchements inutiles ou les retours non désirés. Grâce à Deepseek, la solution a consisté à utiliser des « flags » pour configurer correctement la pile, garantissant ainsi un comportement fluide et cohérent lors des transitions entre activités.

Ajout et gestion des musiques

Description : Les utilisateurs peuvent enrichir une liste musicale commune en ajoutant des morceaux personnalisés, avec titre, artiste et genre musical. Les données sont automatiquement récupérées depuis l'API Spotify pour garantir leur précision. Les pochettes des albums, affichées à l'aide de la bibliothèque Picasso, ajoutent une touche visuelle élégante et immersive pour une meilleure navigation.

Défis rencontrés et solutions apportées : Pour éviter les doublons, une vérification des IDs uniques des morceaux a été mise en place avant chaque ajout. Si un morceau existe déjà, l'utilisateur est immédiatement informé via un message Toast clair et informatif⁷

```
private void addMusicToFirestore(Music music) {
    firestoreRepository.getFirestore().collection( collectionPath: "proposals") CollectionReference
        .whereEqualTo( field: "id", music.getId()) Query
        .get() Task<QuerySnapshot>
```

Par ailleurs, un problème technique avec Picasso provoquait un crash de l'application lors du chargement des images. Une solution efficace a été d'encadrer ce processus avec un bloc try-catch, permettant de gérer les erreurs et d'utiliser une image par défaut en cas d'échec. Cela

```
try {
    Picasso picasso = new Picasso.Builder( context: this)
        .indicatorsEnabled(false)
        .loggingEnabled(BuildConfig.DEBUG)
        .build();
    Picasso.setSingletonInstance(picasso);
} catch (IllegalStateException e) {
    Log.d(TAG, msg: "Picasso already initialized");
}
```

garantit une continuité visuelle sans compromettre la stabilité de l'application. Un autre défi concernait la récupération du genre musical. L'API Spotify associe les genres aux artistes et non directement aux morceaux. En analysant les réponses JSON via Postman⁸, une solution a été trouvée : capturer le premier genre de l'artiste dans l'array JSON et le stocker dans

⁶ Ce code se trouve dans le répertoire : « ...\\musio\\main\\MainActivity » L : 107

⁷ Voir les codes dans le répertoire : « ...\\musio\\main\\MusicActivity » L : 235

⁸ Postman est l'outil que j'ai utilisé pour tester les API en ligne [Postman](https://www.postman.com/) | [Postman API Network](https://www.postman.com/api-network/)

Firestore avec les informations de la musique. Cela assure une cohérence dans la gestion des genres musicaux.

Gestion centralisée avec FirestoreRepository

Description : Pour simplifier et optimiser les interactions avec Cloud Firestore, la classe **FirestoreRepository** a été développée. Elle agit comme une couche intermédiaire centralisant toutes les opérations liées à Firestore, telles que l'ajout, la lecture, la modification et la suppression de documents. Cette approche améliore l'organisation du code et facilite sa maintenance et son évolution.

Défis rencontrés et solutions apportées : Initialement, les appels à Firestore étaient dispersés dans plusieurs classes ou activités, rendant le code difficile à lire et à gérer. Cette fragmentation augmentait également les risques d'erreurs, comme les conflits entre requêtes simultanées. La solution proposée par **Deepseek** a été de regrouper toutes les interactions avec Firestore dans une seule classe : **FirestoreRepository**.

Avantages principaux :

- ✓ **Organisation claire** : Toutes les opérations Firestore sont encapsulées dans une seule classe, ce qui simplifie la gestion du code et améliore sa lisibilité.
- ✓ **Réutilisabilité** : Les méthodes de FirestoreRepository (comme « addMusic() », « getUserData() », « updateLikes() ») peuvent être appelées par différentes activités ou fragments sans duplication de code.
- ✓ **Gestion des erreurs** : La classe inclut une gestion unifiée des erreurs pour éviter les plantages imprévus ou les comportements incohérents.
- ✓ **Performances optimisées** : En regroupant les appels et en rationalisant les requêtes, FirestoreRepository réduit les requêtes redondantes, augmentant ainsi la réactivité de l'application.

La mise en œuvre de FirestoreRepository a non seulement simplifié l'intégration de Firestore, mais elle a également permis une évolution future plus flexible. Cette structure modulaire facilite l'ajout de nouvelles fonctionnalités liées aux données, tout en réduisant les risques d'erreurs.

Jouer une musique via l'application

Description : L'API Spotify permet de jouer des morceaux, mais impose certaines contraintes :

- Le développeur doit disposer d'un abonnement Spotify Premium.
- L'utilisateur doit avoir l'application Spotify installée et être connecté à son compte.

Pour contourner ces limites, une requête YouTube est générée à partir du titre et de l'artiste, redirigeant l'utilisateur vers YouTube pour écouter la musique.

Défis rencontrés et solutions apportées : Ces contraintes ont limité l'intégration directe de Spotify. La redirection vers YouTube a offert une solution accessible, contournant les restrictions tout en maintenant une expérience utilisateur simple.

Redirection vers YouTube

Description : La fonctionnalité de redirection vers YouTube a été intégrée pour permettre aux utilisateurs d'écouter les morceaux sélectionnés via une recherche YouTube dynamique.

Musio génère une requête de recherche à l'aide de la méthode utilitaire `generateYouTubeUrl()`⁹ :

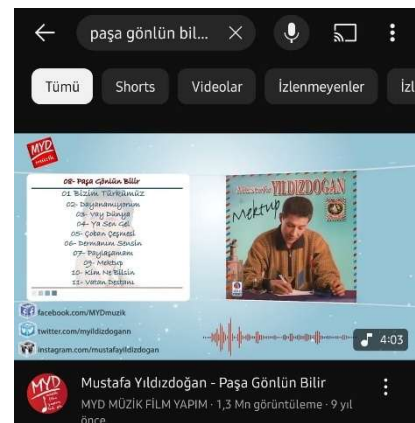
```
public static String generateYouTubeUrl(String trackName, String artistName) {  
    String query = Uri.encode(trackName + " " + artistName);  
    return "https://www.youtube.com/results?search_query=" + query + "&sp=EgIQAQ%3D%3D";  
}
```

Cette URL est ensuite exploitée dans `MainActivity` et dans `MusicActivity`¹⁰ pour rediriger l'utilisateur à l'aide d'un `Intent` standard :

```
@Override  
public void onPlayClick(String trackName, String artistName) {  
    String youtubeUrl = MusicUtils.generateYouTubeUrl(trackName, artistName);  
    startActivity(new Intent(Intent.ACTION_VIEW, Uri.parse(youtubeUrl)));  
}
```

Défis rencontrés et solutions apportées : Musio devait contourner une contrainte imposée par l'API Spotify : un abonnement Premium est requis pour intégrer la lecture de morceaux directement dans l'application. Afin de surmonter cette limitation, la redirection vers YouTube a été mise en place, offrant aux utilisateurs une alternative gratuite et accessible.

En centralisant la logique dans la classe utilitaire `MusicUtils`, les risques d'erreurs ont été considérablement réduits. Par exemple, même si l'application YouTube n'est pas installée sur l'appareil de l'utilisateur, le système Android redirige automatiquement l'URL vers le navigateur par défaut, assurant ainsi une continuité de la fonctionnalité.



Grâce au comportement natif d'Android, lorsque l'utilisateur interagit avec un lien YouTube généré, celui-ci est ouvert soit dans l'application YouTube (si elle est disponible), soit dans le navigateur. Cette gestion intelligente garantit une expérience utilisateur fluide, même en cas de configurations variables des appareils.

Like des musiques

Description : Les utilisateurs peuvent liker leurs morceaux préférés une seule fois par musique. Lorsqu'un utilisateur clique sur le bouton ♥, son identifiant est enregistré dans Firestore avec la musique. Si le morceau est déjà liké, l'icône ♥ devient rouge. En re cliquant,

⁹ Ce code se trouve dans le répertoire : « ...\\musio\\main\\utils\\MusicUtils » L : 18

¹⁰ Ce code se trouve dans le répertoire « ...\\musio\\main\\MusicActivity » L : 266 et L 405 dans `MainActivity`

le like est annulé, l'identifiant est retiré de Firestore, le compteur de likes est décrémenté, et l'icône revient à sa couleur initiale (voir la partie UX/UI).

Défis rencontrés et solutions apportées : La gestion des interactions multiples et la synchronisation en temps réel avec Firestore ont permis de construire un système de likes fluide et intuitif, comme détaillé dans les défis précédents.

Cartographier les genres musicaux

Description : Cette fonctionnalité géolocalise les genres musicaux sur une carte, avec des marqueurs colorés préconfigurés via BitmapDescriptorFactory de Google Maps. Lors de l'ajout d'une musique, l'utilisateur accepte de partager sa position, qui est enregistrée dans Firestore. Le marqueur correspondant s'affiche automatiquement sur la carte, permettant par exemple d'observer que le Solbosch est principalement associé au rock.

Défis rencontrés et solutions apportées : L'utilisation de BitmapDescriptorFactory a évité la conception manuelle des icônes, un gain de temps par rapport à OSM ou Mapbox, et a permis une intégration fluide et intuitive des marqueurs

Réinitialisation automatique de la playlist

Description : La playlist est réinitialisée toutes les 24 heures : son contenu est supprimé dans Firestore. Une notification cliquable informe l'utilisateur qu'une nouvelle playlist est prête et lui permet d'y accéder directement.

Défis rencontrés et solutions apportées : La réinitialisation et la notification ont été automatisées grâce à Deepseek, rendant le processus fluide et sans intervention manuelle.

Mise à jour en temps réel de la playlist

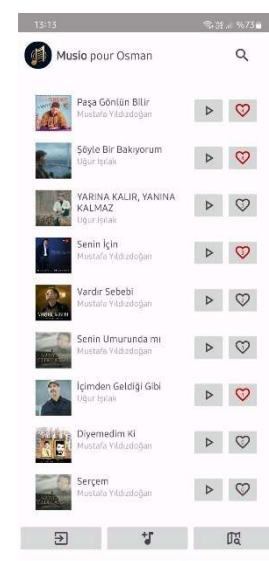
Description : Lorsqu'un utilisateur ajoute une musique, l'ajout est instantanément visible pour les autres, grâce aux listeners en temps réel de Firestore. Par exemple, si Bob ajoute une musique, Alice la voit immédiatement sans rafraîchir l'application.

Défis rencontrés et solutions apportées : La synchronisation en temps réel a été mise en œuvre grâce aux listeners Firestore, comme expliqué dans les défis précédents.

Classement dynamique des musiques par nombre de likes

Description : Les musiques sont automatiquement classées du plus liké au moins liké. Grâce à la mise à jour en temps réel, le classement change instantanément lorsqu'un utilisateur ajoute ou retire un like, permettant de suivre en direct l'évolution des morceaux populaires.

Défis rencontrés et solutions apportées : Les défis rencontrés incluent l'utilisation des listeners Firestore et la gestion des interactions, comme mentionné précédemment.



Analyse

L'application **Musio** repose sur une architecture modulaire respectant les principes SOLID¹¹, où chaque couche remplit un rôle spécifique, interconnecté pour assurer une expérience fluide. Cette analyse explore les choix techniques et les relations entre ces composants.

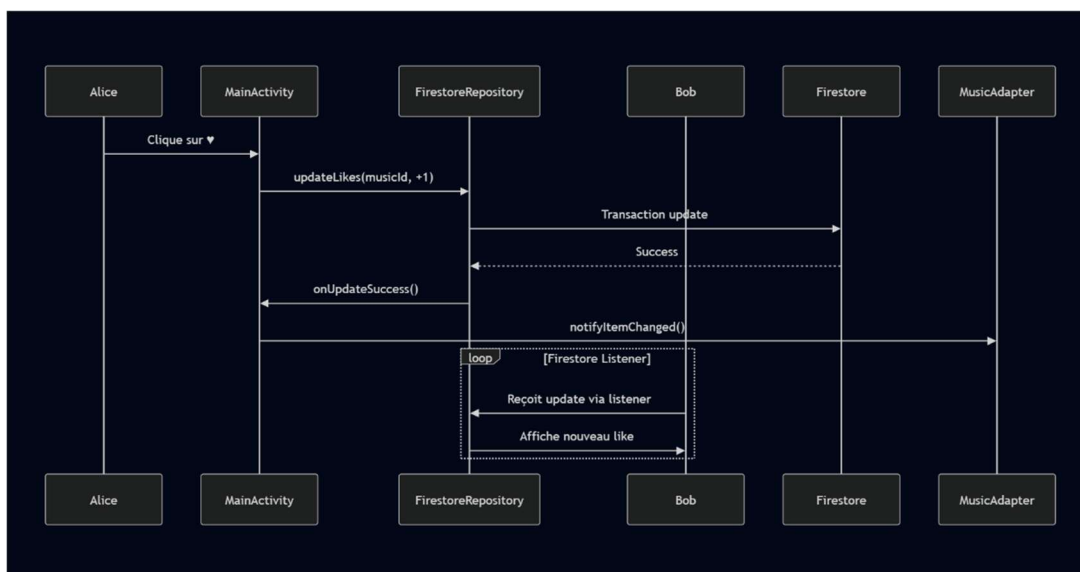
Architecture globale

L'application est organisée en quatre couches principales :

- I. **Couche présentation** : Gère l'interface utilisateur et transmet les actions utilisateur aux couches sous-jacentes.
- II. **Couche données** : Centralisée par FirestoreRepository, elle assure une gestion fiable des informations via Firestore.
- III. **Services autonomes** : Automatisent les tâches périodiques, avec une exécution précise orchestrée par PlaylistService et PlaylistCleanupReceiver.
- IV. **Utilitaires transversaux** : Regroupent des fonctions réutilisables via des classes comme MusicUtils et NotificationUtils

Interactions utilisateur : Cas concret du like

Lorsqu'Alice clique sur ♥ pour liker une musique, l'interaction est immédiatement captée par MainActivity, qui agit comme contrôleur. La demande est transmise à FirestoreRepository via la méthode updateLikes(), où une transaction sécurisée est exécutée dans Firestore pour modifier de manière fiable le compteur de likes. Une fois la mise à jour réussie, MainActivity utilise MusicAdapter pour rafraîchir l'affichage. Pendant ce temps, les listeners en temps réel de Firestore synchronisent immédiatement la modification, permettant à Bob, un utilisateur connecté, de voir le compteur actualisé sur son interface. Ce flux garantit une expérience fluide et engageante. Le flux d'interaction entre Alice, Bob et les composants de l'application est illustré par le diagramme suivant :



¹¹ Principes SOLID visent à structurer un code clair, modulaire et flexible en séparant les responsabilités, facilitant l'extension, garantissant l'interopérabilité, adaptant les interfaces aux besoins et réduisant le couplage des dépendances. [SOLID : les 5 premiers principes de conception orientée objet | DigitalOcean](#)

Gestion des tâches périodiques

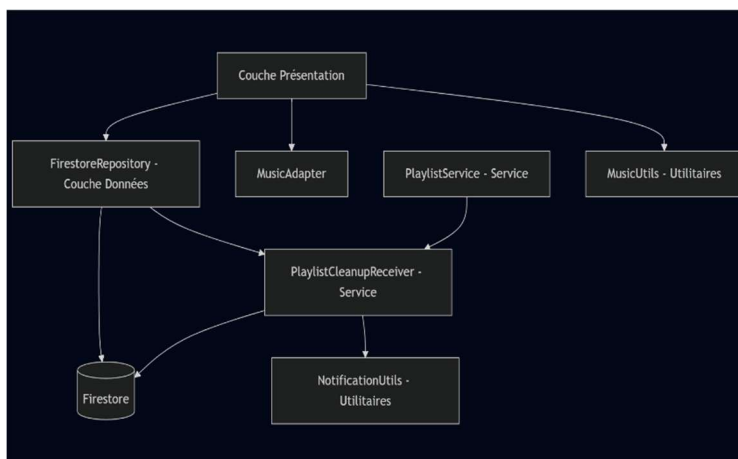
La gestion des tâches périodiques est orchestrée entre plusieurs composants. PlaylistService planifie les nettoyages réguliers grâce à AlarmManager, qui assure une exécution fiable, même en arrière-plan. À l'heure programmée, PlaylistCleanupReceiver se déclenche pour effectuer le nettoyage en appelant FirestoreRepository, qui supprime les données obsolètes directement dans Firestore. Une fois le nettoyage terminé, NotificationUtils informe l'utilisateur via une notification, offrant une confirmation claire. Ce processus met en lumière une séparation nette des responsabilités : PlaylistService planifie, PlaylistCleanupReceiver exécute, et FirestoreRepository gère les données

Le fonctionnement de la gestion des tâches est schématisé ci-dessous :



Relations entre les couches de l'architecture

Les composants de l'application sont organisés en couches distinctes mais interdépendantes. Le diagramme ci-dessous illustre les relations entre les différentes couches de l'architecture :



La **couche présentation** agit comme une interface pour les utilisateurs.

La **couche donnée** centralise les interactions avec Firestore.

Les **services** automatisent les processus périodiques.

Les **utilitaires** apportent modularité et réutilisabilité.

Ces interactions mettent en évidence la coopération entre les différentes classes qui composent l'architecture de **Musio**. Par exemple, MainActivity se charge d'intercepter les actions utilisateur, tandis que MusicAdapter facilite l'affichage dynamique des données. Du côté des services, PlaylistService coordonne les tâches programmées et délègue leur exécution à PlaylistCleanupReceiver, qui travaille en collaboration avec FirestoreRepository pour maintenir l'intégrité des données. Enfin, les utilitaires comme MusicUtils simplifient les opérations liées aux données musicales et NotificationUtils assure une communication fluide avec les utilisateurs via des notifications. Cette interconnexion garantit une gestion efficace des processus et une expérience utilisateur optimisée.

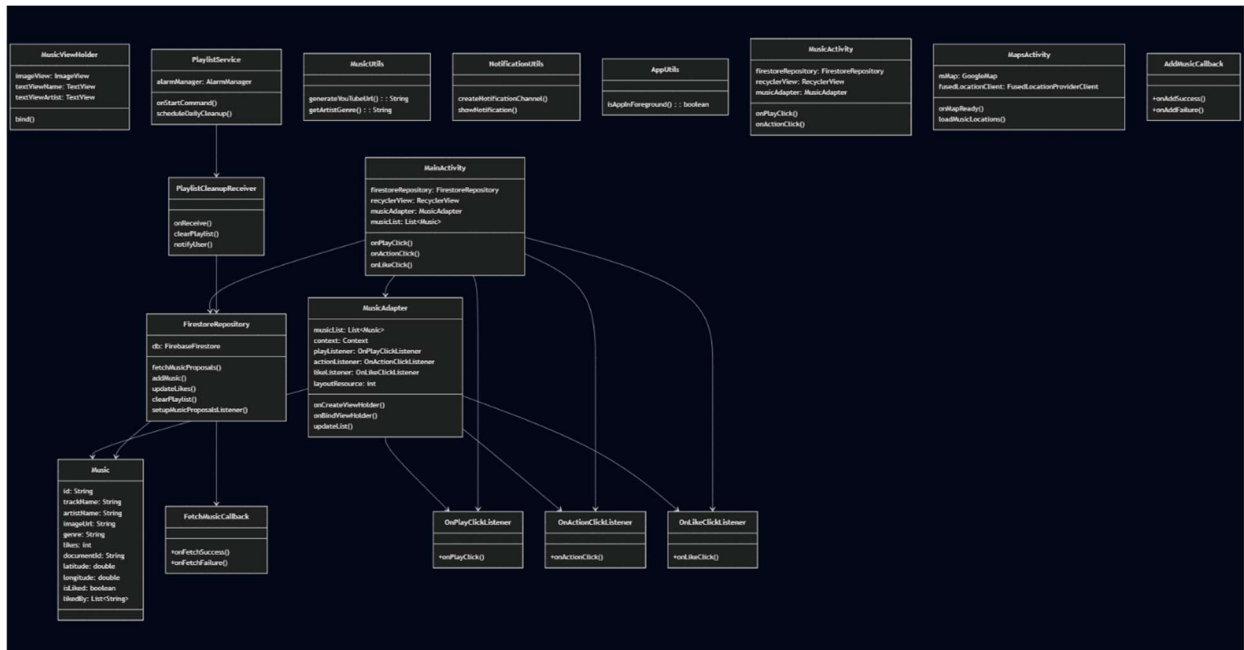
Vue d'ensemble avec le diagramme de classes principal

Le diagramme global présenté ci-dessous détaille les relations entre toutes les classes de l'application ainsi que leurs responsabilités. Il met en lumière l'organisation modulaire de l'architecture et l'interconnexion efficace entre les différents composants.

Vue d'ensemble avec le diagramme de classes principal

Le diagramme global ci-dessous détaille les relations entre toutes les classes de l'application, depuis l'interface utilisateur jusqu'à la gestion des données. Ce schéma met en avant l'organisation modulaire et l'interconnexion des différentes parties.

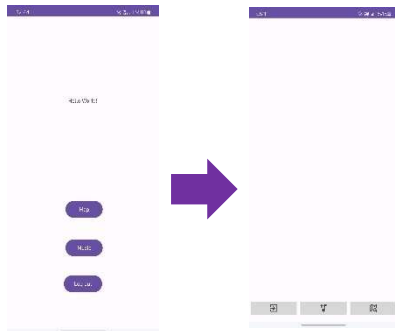
Voici un diagramme global montrant la structure complète de l'application et les relations entre ses composants :



L'analyse révèle la robustesse et la modularité de l'architecture de Musio. L'interconnexion efficace de ses composants permet une expérience utilisateur fluide et cohérente. La séparation des couches, alliée à une gestion centralisée des données et une automatisation fiable des processus, garantit une maintenance simplifiée et une évolutivité future. Les diagrammes inclus renforcent cette compréhension en offrant une visualisation claire des relations et des flux au sein de l'application. En respectant des principes solides et des pratiques Android éprouvées, **Musio** s'affirme comme une solution bien pensée, prête à évoluer.

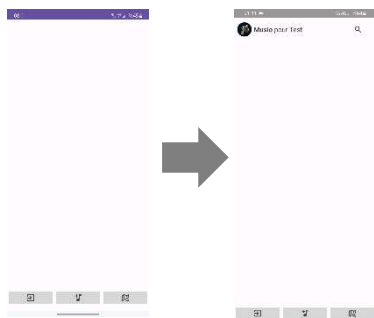
UX/UI

ImageButton pour une interface intuitive



Les boutons classiques ont été remplacés par des **ImageButton** pour améliorer l'expérience utilisateur. Grâce à leurs icônes¹², les actions sont immédiatement compréhensibles, ce qui simplifie la navigation tout en apportant une esthétique moderne et épurée. Ce choix renforce l'engagement visuel et répond aux attentes des utilisateurs en matière de design contemporain.

SearchView et en-tête personnalisée

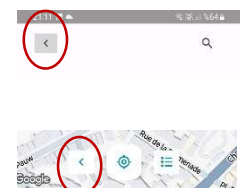


Un **SearchView** a été ajouté dans l'en-tête de MainActivity pour permettre aux utilisateurs de rechercher rapidement une musique dans la playlist. Cela facilite la navigation, surtout lorsque la liste est longue.

Le logo de l'application et le titre personnalisé, comme *Musio pour [NomUtilisateur]*, ajoutent une touche personnelle et rendent l'interface plus accueillante et moderne. Cette personnalisation crée un lien direct avec l'utilisateur tout en renforçant l'identité visuelle de l'application.

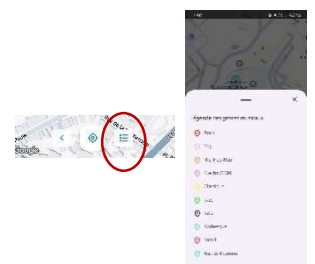
Bouton retour dans MusicActivity et MapsActivity

Un bouton retour a été ajouté dans les activités **MusicActivity** et **MapsActivity** pour permettre aux utilisateurs de revenir facilement à **MainActivity**. Ce choix améliore la navigation en offrant une transition fluide entre les écrans et garantit une expérience utilisateur intuitive. Il réduit les risques de confusion et facilite l'accès aux fonctionnalités principales de l'application.



Légende dans MapsActivity

Une légende sous forme de **Bottom Sheet Dialog**¹³ a été ajoutée dans **MapsActivity** pour permettre aux utilisateurs d'identifier les genres musicaux associés aux couleurs des marqueurs. Cela rend la fonctionnalité plus intuitive en offrant une explication claire, tout en conservant une présentation épurée et accessible.



¹² Les icônes proviennent de Google Fonts : [Material Symbols & Icons - Google Fonts](#)

¹³ Bottom Sheet Dialog inspiré du tutoriel : [Bottom Sheet Dialog in Android Studio using Bottom Navigation Bar | Floating Action Button | 2024](#)

Mode jour/nuit automatique

L'application intègre un **mode jour/nuit** qui s'adapte automatiquement au mode activé sur l'appareil mobile. Cette fonctionnalité améliore le confort visuel des utilisateurs en ajustant les couleurs et l'apparence de l'interface selon les conditions d'éclairage. Le mode jour offre une meilleure visibilité en lumière vive, tandis que le mode nuit réduit la fatigue oculaire dans des environnements peu éclairés.

Système de like interactif

Lorsqu'un utilisateur aime une musique, le compteur de likes s'incrémente et l'icône ♥ devient rouge pour signaler l'action. En cliquant à nouveau, l'utilisateur annule son like : le compteur est décrémenté et l'icône retourne à son état initial.



Feedback de l'étudiante

Une étudiante de 1^{ère} année, passionnée et curieuse, a partagé son expérience :

Points positifs : Elle a particulièrement apprécié le détail personnalisé dans l'en-tête, « *Musio pour elle* », qui crée une expérience plus accueillante. La fluidité et la simplicité des icônes, ainsi que le système interactif de likes et la redirection vers YouTube, ont été des points forts pour elle. Elle a également trouvé l'intégration du mode jour/nuit très pratique pour le confort visuel.

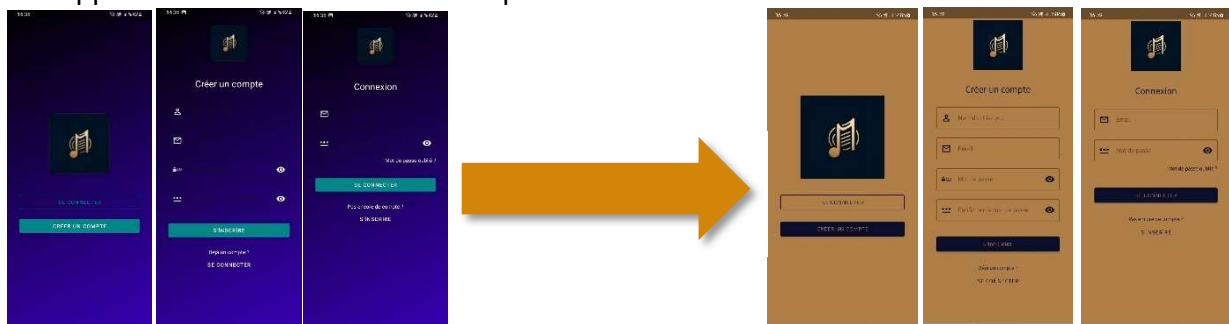
Points négatifs : Elle a suggéré de rendre **MainActivity** et **MusicActivity** plus engageantes, car elles peuvent paraître trop vides. Elle recommande de revoir les couleurs dans la partie connexion pour une meilleure harmonie visuelle. Elle aurait aussi aimé que **MusicActivity** propose des musiques au hasard, pour enrichir l'expérience utilisateur.

Dans l'ensemble, elle a conclu qu'elle a aimé l'application et a trouvé l'expérience utilisateur plaisante et bien pensée.

Améliorations après le feedback de l'étudiante

Révision des couleurs dans la partie connexion

Les couleurs de la partie connexion, avec un fond jaune doré et des boutons et textes en bleu foncé, ont été soigneusement choisies pour respecter la charte graphique du logo. Un fond jaune doré a été choisi pour apporter une touche chaleureuse et accueillante, tandis que les boutons et textes en bleu foncé créent un contraste élégant et professionnel. Cette combinaison améliore non seulement la lisibilité, mais renforce également l'identité visuelle de l'application en offrant une esthétique moderne et harmonieuse.



Améliorations apportées à MainActivity et MusicActivity

Pour MainActivity :

Lorsqu'une playlist est vide, un message s'affiche sur le **RecyclerView**, informant l'utilisateur que la playlist est vide et l'invitant à en ajouter une. Cela évite de laisser une page vide et guide l'utilisateur de manière proactive.

Lorsqu'une recherche via le **SearchView**¹⁴ ne donne aucun résultat, un message clair, "*aucun résultat*", est affiché. Cela améliore la transparence et l'expérience utilisateur.

Pour MusicActivity :

Un message s'affiche sur le **RecyclerView**¹⁵ encourageant l'utilisateur à cliquer sur l'icône de recherche pour trouver une musique. Cela oriente l'utilisateur et remplit visuellement l'interface.

Pour les deux activités :

Un **ProgressBar** doré est désormais affiché pendant le chargement des informations, apportant une touche esthétique cohérente avec le design général et améliorant l'expérience utilisateur en signalant que le système est en cours de traitement.

Page MainActivity : le cas avec la playlist vide	Page MainActivity : le cas où la recherche ne donne aucun résultat	Page MusicActivity	Le ProgressBar en action

¹⁴ SearchView est un widget Android qui permet aux utilisateurs de rechercher rapidement du contenu dans une application.

¹⁵ RecyclerView est un widget Android pour afficher des listes efficacement. On l'utilise pour l'affichage des musiques

Limitations et développement futur

Limitations

Connexion Internet

L'application nécessite une connexion Internet stable pour fonctionner. Elle s'appuie sur des services comme Spotify, Cloud Firestore, Firebase Authentication et Google Maps pour récupérer et traiter des données en temps réel. En l'absence de réseau, les fonctionnalités principales deviennent inaccessibles, ce qui peut nuire à l'expérience utilisateur dans les zones où la connectivité est limitée.

Limitations des API

Cloud Firestore : Les quotas gratuits imposent des limites : jusqu'à 50 000 lectures, 20 000 écritures, et 10 Go de transfert de données par jour. Cela limite les actions possibles : si chaque utilisateur effectue en moyenne 10 lectures par recherche ou navigation, l'application peut supporter environ 5 000 recherches par jour avant d'atteindre la limite. La taille maximale des documents (1 Mo) exige une structuration rigoureuse des données pour éviter les problèmes de stockage. En outre, certaines requêtes avancées (comme OR et NOT) ne sont pas prises en charge, réduisant la flexibilité dans la recherche et le filtrage des données.

Firebase Authentication : Les créations de compte sont limitées à 100 par heure, ce qui peut poser un problème en cas d'afflux massif de nouveaux utilisateurs. Les e-mails de vérification sont plafonnés à 1 000 par jour, ce qui peut bloquer des utilisateurs si la demande dépasse cette limite lors de pics d'inscriptions.

Spotify API : Bien qu'elle permette de récupérer des métadonnées (titres, artistes, albums), la lecture de musique via l'API n'est pas autorisée. Les utilisateurs doivent être redirigés vers YouTube pour écouter les morceaux, ce qui fragmente l'expérience musicale. Par ailleurs, les requêtes sont limitées par des quotas : avec une fenêtre de 30 secondes, cela contraint le nombre d'interactions simultanées si chaque recherche nécessite plusieurs requêtes pour obtenir les détails des morceaux et artistes.

Google Maps API : Avec le crédit gratuit, environ 28 000 chargements de cartes dynamiques par mois sont autorisés, suffisant pour les petites bases d'utilisateurs. Cependant, une base utilisateur importante pourrait rapidement dépasser ce quota, rendant certaines fonctionnalités indisponibles sans passer à une offre payante.

Recommandations et corrections dans la recherche musicale :

La fonctionnalité de recherche musicale ne propose pas encore de recommandations intelligentes basées sur les préférences de l'utilisateur ni de corrections typographiques avancées. Cela peut réduire l'efficacité et la convivialité de la recherche.

Lecture de musique intégrée :

La lecture de musique directement au sein de l'application n'est pas possible pour le moment. À la place, l'utilisateur est redirigé vers YouTube à l'aide d'une requête de recherche avec le nom du morceau et son artiste pour écouter les morceaux sélectionnés. Cette

approche est une solution pratique, mais elle dépend d'une application externe (YouTube ou le navigateur) pour compléter l'expérience musicale.

Bien que ces limitations représentent des défis, elles ouvrent la voie à des améliorations et innovations potentielles dans les futures versions de Musio.

Développements futurs

Proposition de musiques aléatoires sur MusicActivity :

Par suite du retour de l'étudiante, une section dédiée aux suggestions aléatoires sera envisagée dans MusicActivity. Cette fonctionnalité permettra de proposer une sélection de morceaux aléatoires pour enrichir l'expérience utilisateur et rendre l'application plus interactive.

Un bouton "Découvrir" pourrait être ajouté, morceaux offrant à l'utilisateur la possibilité de rafraîchir les suggestions et d'explorer de nouveaux à chaque interaction.

Les recommandations seraient basées sur des tendances générales ou des données aléatoires issues d'une base disponible via les API, tout en respectant les limitations techniques actuelles.

Notification de la musique la plus votée avant réinitialisation

Avant chaque réinitialisation de la playlist, l'application pourrait notifier l'utilisateur de la musique la plus votée de la veille. Cela met en valeur les morceaux populaires, encourage l'engagement, et préserve une trace des préférences des utilisateurs.

Affichage de la musique de la semaine ou du mois

Une fonctionnalité dédiée pourrait afficher la musique la plus populaire de la semaine ou du mois. Cette section mettrait en avant les tendances sur une période donnée et offrirait aux utilisateurs une perspective globale des morceaux les plus appréciés. Pour rendre cette mise en avant encore plus attrayante, un icône de trophée ou de médaille pourrait être ajouté devant la pochette de l'album, symbolisant ainsi la reconnaissance du morceau le plus populaire. Cela renforcerait l'impact visuel et motiverait les utilisateurs à participer davantage.

Exploration d'une solution avec une heatmap

Comme mentionné lors de la revue de la deuxième itération, une heatmap pourrait être utilisée pour enrichir la représentation des genres musicaux. Chaque genre musical serait associé à une couleur unique, permettant de visualiser facilement les zones où ce genre est populaire. Cette approche remplacerait l'utilisation de marqueurs isolés en offrant une vue d'ensemble plus intuitive et plus esthétique des associations géographiques aux genres musicaux.

Cependant, cette fonctionnalité était discutée comme facultative et n'était pas une exigence obligatoire. Son intégration pourrait être envisagée dans le futur, en fonction des priorités et des ressources disponibles.

Ces évolutions permettront non seulement d'améliorer l'expérience utilisateur, mais également de renforcer l'engagement et la fidélité au sein de la communauté Musio.

Conclusion

Ce projet m'a plongé dans un domaine totalement nouveau pour moi, ce qui a rendu l'expérience d'autant plus intéressante et enrichissante. Avoir l'opportunité de concevoir une application selon mes propres idées, au lieu de travailler sur un projet imposé, a véritablement amplifié ma motivation et ma curiosité. Cela m'a permis d'apprendre en expérimentant, tout en affrontant les défis techniques qui ont marqué mon parcours.

Concernant l'intelligence artificielle, dans mon rapport de Web III, j'avais exprimé certaines réserves, en soulignant que l'IA pouvait parfois induire en erreur et qu'il était essentiel d'analyser ses suggestions avec un regard critique. Ces points restent valables : elle n'est pas toujours fiable, mais dans ce projet mobile, elle s'est avérée indispensable. Lorsque la documentation était obsolète ou insuffisante, l'IA m'a permis de surmonter les obstacles, comme résoudre le problème des autorisations par défaut dans Firestore Repository qui m'a bloqué une semaine. La solution, passer de « false » à « true », m'a été indiquée grâce à l'IA, ce qui m'a permis de progresser dans le développement.

En plus des fonctionnalités intégrées, j'ai trouvé très pertinente l'idée de l'étudiante d'ajouter une option permettant de proposer des musiques aléatoires. Bien que cette fonctionnalité n'ait pas été mise en place dans cette version, elle représente une piste intéressante pour de futures améliorations.

Si je devais améliorer ce projet, je travaillerais sur plusieurs aspects, notamment :

- ✓ Une meilleure optimisation de l'expérience utilisateur et de l'interface (UX/UI), afin de garantir une utilisation fluide et intuitive.
- ✓ L'ajout de fonctionnalités supplémentaires, comme l'actualisation en temps réel des données ou une gestion améliorée des performances pour réduire les temps de réponse.
- ✓ Une visualisation encore plus sophistiquée, avec une heatmap permettant de représenter les genres musicaux de manière claire et interactive.

Bien que l'évaluation soit un aspect du projet, je préfère mettre en avant l'opportunité unique qui m'a été donnée : celle de concrétiser une idée personnelle dans le cadre de l'apprentissage. Cela ne m'a pas seulement permis d'apprendre, mais m'a aussi apporté une dimension positive qui a nourri ma créativité et m'a permis de donner vie à **Musio**. Cette application est le résultat d'un parcours riche en exploration, en défis surmontés, et en évolution personnelle.

Mais au-delà de ces considérations techniques et de l'évaluation, ce projet a été une véritable aventure. Le fait de pouvoir transformer une idée personnelle en application fonctionnelle a été une expérience marquante. Cela m'a permis de développer mes compétences tout en renforçant ma créativité et ma motivation. Je suis fier d'avoir pu donner vie à **Musio**, et je considère cette expérience comme une réussite. Elle restera une

étape clé dans mon parcours, m'aidant à évoluer et à aborder des défis encore plus ambitieux à l'avenir.

Liens YouTube vers les vidéos

Vidéo de démonstration

https://youtu.be/mCfVu1S_4ko

Vidéo Bonus

<https://youtu.be/WjtGE5rsP1s>